,,

LENGUAJES Y AUTÓMATAS II

RESOLVER PROBLEMAS

No se van a definir el concepto de **“Problema”,** ya que esto es muy complejo y se ve en temas de filosofía. Aun así, se podría definir como una cuestión difícil de solucionar, es decir que no tiene una solución trivial.

Todos recordaran el paso por los semestres anteriores y por la primaria donde se tenia que resolver problemas matemáticos, así que vamos a recordar 2 problemas

Problema 1: Eduardo tiene 5 paletas. De camino a su casa se le cae una sin que se diera cuenta, al llegar a su casa en las escaleras pierde otra paleta, al entrar a su cuarto se encuentra 3 paletas mas en la mesa de su hermano Kevin. ¿Cuantas paletas tiene ahora Eduardo?

Problema: Ordenar en orden descendente la siguiente lista de números: 3,6,1,3,9,4,2

Son problemas muy simples pero como se enfrenta nuestro cerebro a ello si se ha intentado encontrar la solución de ambos. Es inherente darse cuenta que existen diferentes formas de solucionarlo, a pesar que son de baja dificultad.

El primer problema, seguramente se ha intentado contextualizar en su situación particular. Seguramente se han imaginado a Eduardo perdiendo sus paletas y encontrando algunas mas. Y al final se realizar una simple operación de resta y suma.

Paletas = 5-1-1+3 = 6

El segundo es un poco distinto ya que la tarea consiste en evaluar mediante comparaciones lógicas y así podemos saber cual va primero y cual va después.

En ambos casos nuestro cerebro ha recolectado información y le ha dado contexto, el cual es necesario para olvidar todo aquello que genere ruido y traducido a un modelo mental que es capaz de resolver.

Sin embargo no todos los problemas son sencillos. Es necesario plantear algunos problemas algo mas divertidos, los cuales sentaran la base para seguir indagando en la solución de problemas.

PROBLEMAS COMPLEJOS

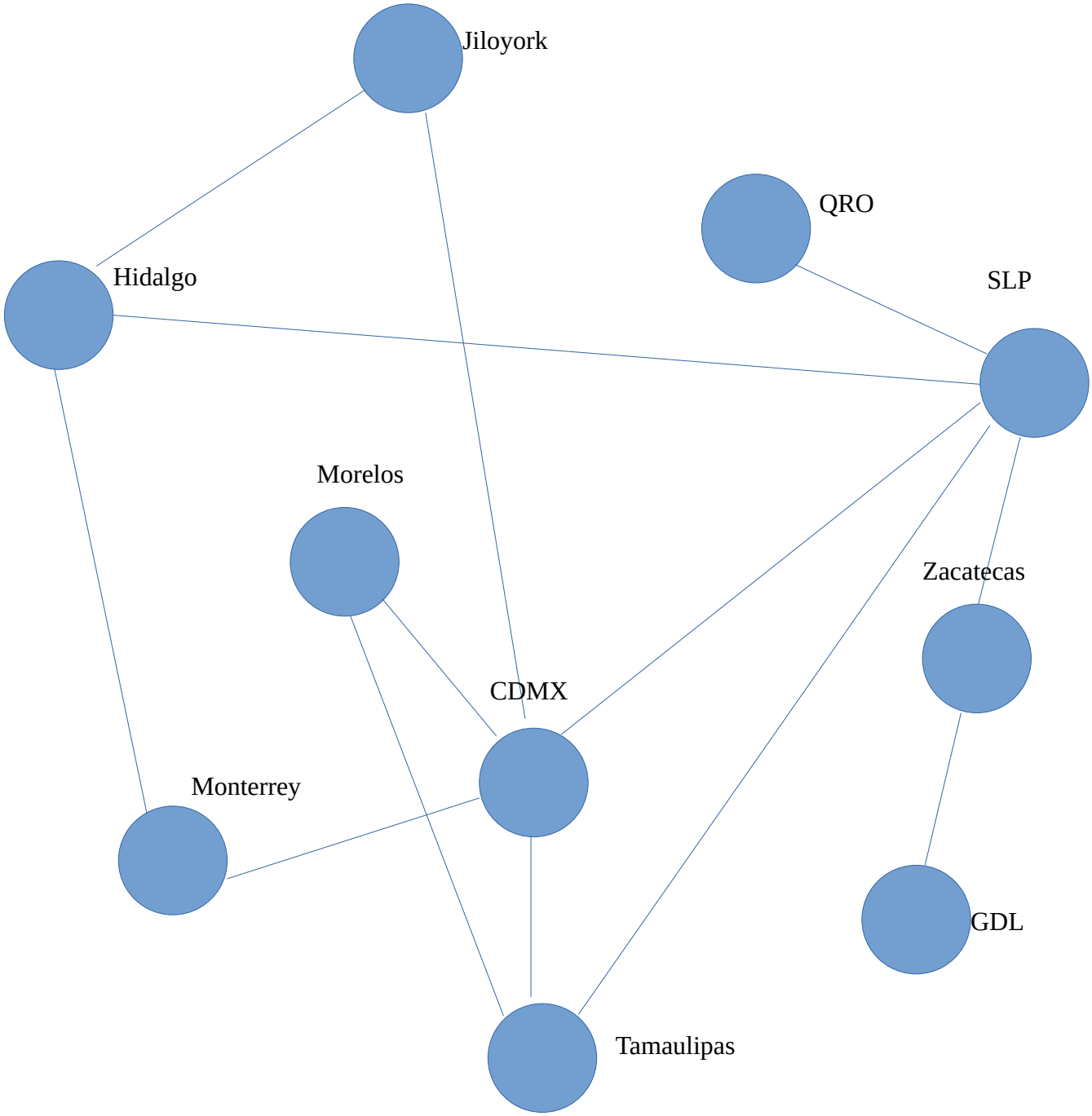
Es necesario describir algunos problemas que servirán de modelo a seguir durante el resto del curso

Si ustedes han echo algún viaje largo, ya sea en autobús o en avión, seguramente han intentado viajar al destino directamente y sin escalas. Este problema es clásico y con una gran aplicación en la vida real. El problema podría describirse de la siguiente manera: dada una ciudad origen con su ciudad destino, encontrar una combinación de vuelos que permita

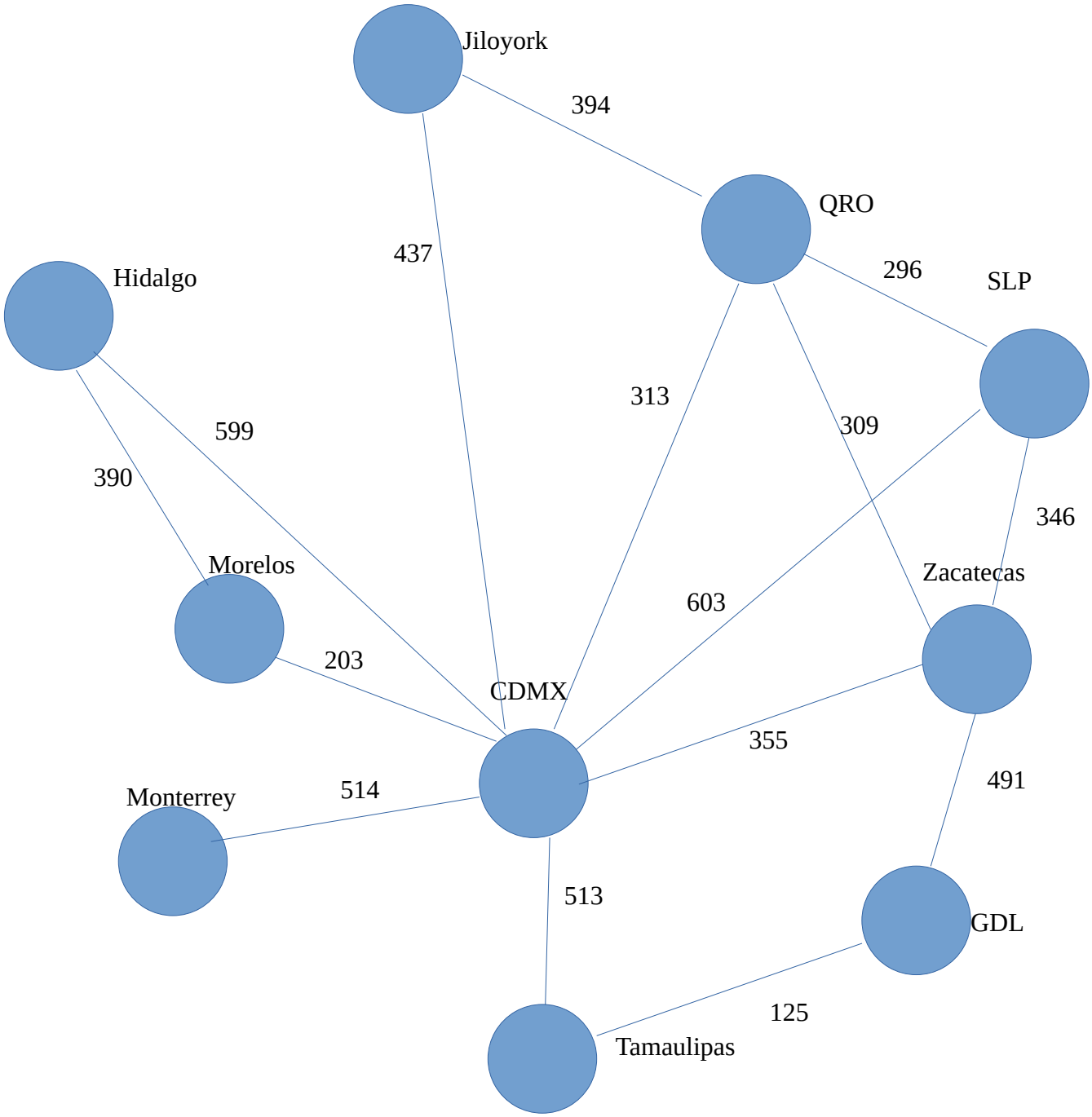
cubrir la distancia con el mínimo de transbordos. Por ahora tiene sentido obviar en tiempos de vuelo, distancia, horarios de vuelo y todo lo que conlleva viajar.

Ahora vamos a en listar otro problema en esta ocasión se trata de ir de una ciudad a otra, pero por carretera, tratando de realizar el menor numero de kilómetros posibles. Parece que es un problema similar al de los aviones, solo que en lugar de pasar por aeropuertos, se pasa por diversas ciudades, sin embargo hay un pequeño detalle que hace que el problema sea diferente. Y es que los objetivos son distintos.

En el primer problema se tenia como objetivo: minimizar el numero de transbordos. En el segundo se trata de minimizar la distancia por carretera.



Supongase que se desea ir de Tamaulipas a Hidalgo según el criterio es el de relizar el numero mínimo de transbordos si se quisiera viajar de Tamaulipas a Hidalgo, pero en esta ocasión por carretera el objetivo es el de minimizar las distancias recorridas



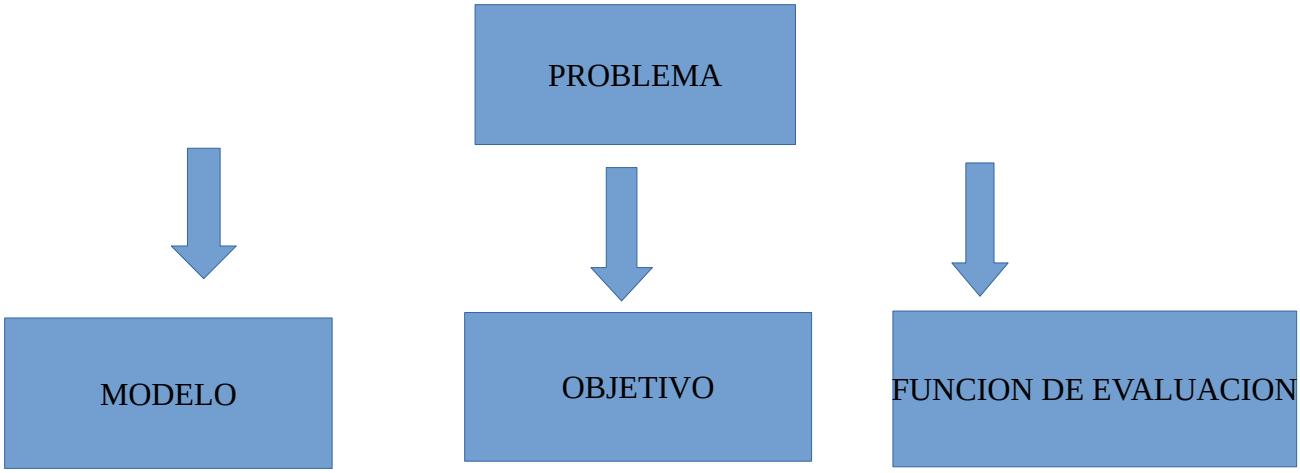
Supongamos que ahora se quiere ir de la ciudad de Tamaulipas a Hidalgo estas serian las posibles opciones:

TAMAULIPAS > CDMX > HIDALGO (1,112 km)
TAMAULIPAS > CDMX > MORELOS > HIDALGO (1,106 km)

¿Cual es el mejor recorrido? Se ha definido un criterio que indica que tan bueno y eficiente es un recorrido. En este caso se ha seleccionado la suma total de kilómetros, que arroja el mejor recorrido, el cual nos permite llegar al destino cubriendo una menor distancia.
A simple vista los problemas que se han planteado , se puede intuir que hay 3 conceptos importantes que definen un problema. Por un lado se ha visto que se necesita crear un modelo simplificado que represente el problema real. Por ejemplo: reduciendo el enunciado del problema donde Eduardo y sus paletas a simples operaciones matemáticas.

También se ha llegado a la conclusión de que hay que tener muy claro el objetivo que se busca es decir ¿Cuando se da por resuelto el problema?, ¿Cuáles son las soluciones validas?
En los ejemplos del viaje de avión o por carretera ¿Cuales eran los objetivos? Por ejemplo encontrar las conexiones con el numero de trasbordo y el segundo, encontrar el recorrido con menor numero de km.

Finalmente se ha tenido que definir un criterio que valore como buena cada una de las soluciones del problema para poder compararla con las demás y quedarse con la mejor solucion. En el caso de los viajes de avion, se ha tomado el criterio del numero de aereopuertos visitados. El problema de los viajes por carretera, el criterio la suma total de km. Este criterio da informacion de la calidad de la solucion a lo que a partir de ahora se le llamara **Función de Evaluación**



ALGUNOS TIPOS DE SOLUCIÓN

Antes de empezar a plantear solución a un problema, se ha visto en el apartado anterior, que necesario conceptual izarlo, eliminar información redundante y utilizar estructuras conocidas, como lo son las operaciones matemáticas para lograr moldearlo. El modelo es util para simplificación del modelo real.

Hay que tener muy claro que para construir el modelo se tiene que considerar conseguir la propia solución del problema real.

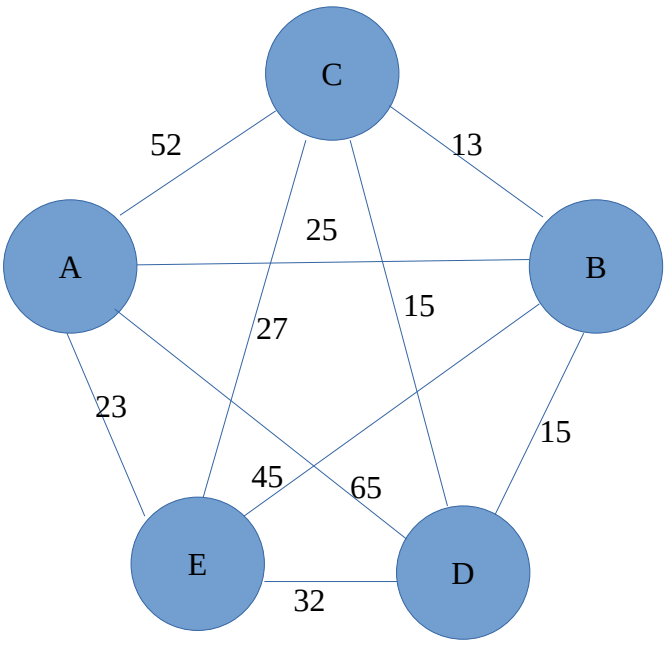
Cuando se conceptual-iza el problema, se necesita una forma de representar la información de la que se dispone, cuando finalmente se dispone, cuando finalmente se haga la implementación en el orden, este modelo tendrá que ser traducido a estructura de datos de operaciones que el ordenador sea capaz de manejar

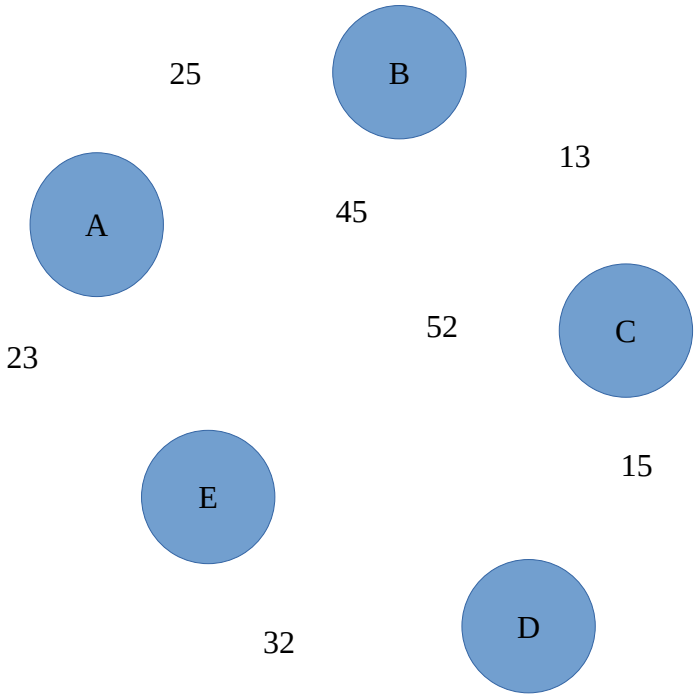
EL PROBLEMA DEL VIAJANTE DE COMERCIO

El problema que se va a tratar es bastante conocido dentro de la IA y es conocido como “El problema del Viajante de Comercio” o también por sus siglas en ingles **TSP (Travelling Salesman Problem).** La formulación del problema es sencilla: sean N ciudades, todas conectadas por carretera entre si, cuya distancia entre ellas es conocida. El objetivo es encontrar una ruta que, empezando por la ciudad y acabando en la misma , pase exactamente una vez por cada una de las ciudades (Excepto por la primera, que será visitada 2 veces) y minimiza la distancia recorrida.

Taras este simple enunciado se esconde un problema tremendamente divertido tanto que incluso los ordenadores más potentes que existen hoy en día no son capaces de resolverlo. Incluso para un numero de ciudades moderado. Aun así, se puede conseguir aproximaciones muy buenas utilizando diferentes técnicas de IA algunas de ellas serán estudiadas en los siguientes temas.

Comencemos por construir un modelo que el ordenador pueda manejar, empecemos por el mapa en si. Hay varias opciones: por ejemplo, se puede optar por representar las ciudades mediante un dígrafo ponderado. En este caso cada nodo del grafo representara una ciudad y cada arista corresponderá a una carretera con su distancia (peso) asociada. Debe tenerse en cuenta que para cada caso el TSP, la distancia a cubrir desde una ciudad A hasta B no tien porque ser la misma de B a A. De todas formas para simplificar el objetivo es necesario suponer que las distancias entre las ciudades son iguales independientemente en el sentido en el que se valla.





Otra posible representación es utilizar una tabla, también llamada matriz de adyacente como la siguiente.

	A	B	C	D	E
A	0	25	52	65	23
B	25	0	13	15	45
C	52	13	0	15	27
D	65	15	15	0	32
E	23	45	27	32	0

Siempre que sea posible se elegirá el la representación más simple e intuitiva, a no ser que en términos de eficiencia sea más congruente utilizar otra estructura de datos mas compleja en caso de que se presenta parece que la tabla es mas interesante ya que puede ser modelada fácilmente con un arreglo de dos dimensiones. La implementación de grafo no aporta nada ya que se trata de una estructura de datos estática que no cambia durante su ejecución. Si la información fuera cambiante durante el tiempo de ejecución, por ejemplo, que se asignen nuevas ciudades entonces si seria interesante usar un grafo como una estructura dinámica.

Una vez modelada la información del problema es necesario una forma de representar la solución, es decir, que el objetivo, pueda ser representado de una forma simple y directa que cumpla con las siguientes premisas:
- La lista comienza y termina por la misma ciudades
- En la lista aparecen todas las ciudades.
- No se repite ninguna ciudad, excepto la primera y la ultima.

Cualquier lista de ciudades que cumpla las tres premisas anteriores es una posible solución.
Sin embargo se sabe que no todas las soluciones son iguales, de echo se busca la ruta con menor distancia. Se necesita una función de evaluación que informe la calidad de solucion para poder compararla con otras soluciones. Para el TSP se usara la suma de as distancias de cada una de las ciudades por laas que va pasando. Por ejemplo, estas son posibles soluciones y el valor obtenido por la solucion.

A – B – E – D – C - A (Función de evaluación: 169)
A – E – D – C – B - A (Función de evaluación:108)
A – C – D – E – B – A (Función de evaluación: 169)

¿Como encontrar la mejor solución? En teoría se podría proceder de la siguiente manera:
- generar todas las posibles soluciones
- Comparar una con otra y quedarse siempre con la que arroje mejor valor

Analicemos con detalle el primer paso. ¿Como se genera todas las soluciones?, ¿Cuantas soluciones hay?

Otra forma de generar todas las posibles soluciones es a partir de una lista de ciudades cualquiera e ir permutandolas entre ellas. Cada permutación serpa una posible solución al problema. Suponga se que la lista de las ciudades es la siguiente:

A – B - E – C - D – A (Función de evaluación: 177)
Si se hace cambio entre las ciudades B y E se obtendrá:
A – E – B – D – C – A (Función de evaluación: 150)

Si se efectúa un cambio de E y D sobre la primera ruta se obtiene:
A – D – E – B – C – A (Función de evaluación: 207)

Y así hasta obtener varias combinaciones posibles pero ¿Cuantas hay? Si se tiene N ciudades, se tiene N! De combinaciones posibles N! = nx(x1) xn(x2)x...x1

5! = 5 *4 * 3 * 2 * 1=120

Con 7 ciudades se tendrían 5,040 posibles soluciones.
Con 10 ciudades se tendrían 3,628,800 posibles soluciones.
Con 20 ciudades se tendrían 2,432,902,008,172,640,000 posibles soluciones.

Para tener una idea, si cada permutación, se le ocupara al ordenador un micro segundo con 20 ciudades se necesaria 77147 años para generar todo el espacio de estados

A este tipo de problemas se le llamará intratables, y se dice que pertenecen a la clase **NP,** que sin entrar en detalles, se dirá que son una clase de problemas cuya complejidad hace que no tenga solución en un tiempo razonable por el ordenador.

Por lo tanto, queda claro que métodos computacionales clásicos no es posible resolver este problema, precisamente la IA trata de como abordar este tipo de problemas a través de diferentes técnicas que serán tratadas en sucesivos temas. Quizá no se pueda crear un código que asegure el mejor resultado, pero si ofrecer un resultado que sea optima o al menos muy buena en un tiempo razonable

El problema de Satisfactibilidad Booleana

Tras este largo y estrombótico nombre, se encuentra otro problema clásico, el nombre de la satis factibilidad booleana o para abreviar **SAT (BOOLEAN Satisfactibility Problem)** tiene le honor de ser el primer problema conocido de la clase **NP,** así, fue solo 4 décadas en 1971 cuando se demostró que SAT era un problema intratable.

Antes de describir el problema se deben recordar conceptos básicos de lógica booleana.
Una variable booleana es aquella que puede tomar dos valores diferentes: Verdadero y Falso. También se puede representar como 1 y 0 o como V y F.

Sobre estas variables se puede realizar varias operaciones pero nos centraremos en las tres principales: Conjunción, Disyunción y Negación.

La Conjunción también conocida como operación AND y representada por el símbolo ^, es una operación binaria en el sentido de que e operan con dos variables y que devueve el valor verdadero solo cuando ambos valores son verdaderos. La siguiente tabla muestra el resultado de operar con la operación conjuntiva (AND).

OP	AND	AP
F	F	F
F	F	V
V	F	F
V	V	V

La disyunción, su operador OR se representa con el símbolo y devuelve el valor verdadero, si el signo de los operadores es verdadero, según la tabla siguiente:

OP	AND	AP
F	F	F
F	V	V
V	V	F
V	V	V

Por ultimo el operador unario opera sobre una sola variable, de negación, operador NOT, se representa con el símbolo ¬

OP	¬OP
F	V
V	F

Es posible construir funciones booleanas, cuyo resultado depende de sus variables por ejemplo:

Si se presenta el valor verdadero como V y Falso como F, las siguientes asignaciones de variables darían como resultado los siguientes valores

∫(*F,V,V)* = (*VF*)¬ V = F
∫(*F,V,V)* = (*VF*)¬ V = V

Las funciones booleanas suelen expresar se de forma normalizada para que sea mas fácil operar con ellas. Las dos formas habituales son: **Forma Normal Conjuntiva (NFC)** y la **Forma Normal Disyuntiva (FND).** La NFC se expresa como agrupaciones de disyunciones llamadas clausulas unidad por conjunciones de la siguiente manera:

(x2 x4)(x3 x1)(x2¬x1)

En cambio, la función FND tiene el siguiente aspecto:

(x2 x4)(x3 x1)(x2¬x1)

Ambas formas son igualmente validad, pero en este caso se usará preferentemente la NFC.

Una clausula puede tener un numero cualquiera de variable, y a su vez una función booleana en NFC o FND pueden tener varias clausulas como sea necesario.

Finalizando este breve recordatorio de lógica booleana en el FNC con n variables, se quiere saber si existe una asignación para las variables sin tener que pensar demasiado.

Por ejemplo la asignación x1 ¿f, x2 = V, x3 = V, x4 = F, satisface la restricciones del problema?

Si ahora se quisiera resolver el problema con una función de tres variables (Problema conocido 3 SAT) y 100 variables booleanas. La cosa se complica un poco. Vamos a plantear un modelo, un objetivo y una función de evaluación.

En este caso el modelo es bastante directo. Se pueden representar asignaciones de las variables como cadenas de valores 0 y 1. por ejemplo x1 = f, x2 = V, x3 = V, x4 = F, estaría representado por una cadena 0110. si se tiene 100 variables, se tendría una cadena de 100 dígitos binarios.

Una cadena cualquiera de n dígitos binarios es una candidata a ser una solución y, por lo tanto, tendríamos que ir evaluándolas como una función de evaluación. Para 100 variables, nuestro espacio de estados (número de posibles soluciones) es de 2100, es un decir, un numero astronómica mente alto. De nuevo nos encontramos con un problema intratable de la clase NP.

Ademas, encontrar una función de evaluación que permita comparar diferentes soluciones tampoco es fácil. Si se evalúa la función, nos devolverá un valor verdadero o falso, pero si no se tiene información de cuantas variables son correctas y cuantas no (Si lo supiéramos , significa entonces que conocemos la solución) Si comparamos dos cadenas diferentes que devuelven el valor falso, ¿como sabemos cual de las dos es la mejor?

Al ser una función NFC, se sabe que la evaluación de todas las cláusulas ha de arrojar un valor verdadero. Si hay uno que no lo es, la función no puede ser verdadera. Por lo tanto, una función de evaluación que nos daría una medida de la calidad de una asignación es aquella que devuelve el numero de clausulas que son verdaderas.

El problema de la programación Lineal Entera

El problema de la programación lineal entera **(PLE)** consiste en la optimización de una función en la que intervienen variables enteras y ademas hay algunas restricciones impuestas, por ejemplo, sea la función:

$\int x1, x2 = 21x1 + 11x2$

Donde x1 y x2 son dos variables enteras y sean también las siguientes restricciones:

$7x1 + 4x2 \leq 13$
$x1 \geq 0$
$x2 \geq 0$

Es decir, se busca una asignación de valores para las dos variables, que maximice o minimice el valor de la función f que a su vez cumpla con las restricciones impuestas de ser mayores o iguales que cero y que la suma de 7x1 y 4x2 sea menor que 13.

Para hacerse una idea de la utilidad que tiene este tipo de problemas se plantea un ejemplo mas elaborado

Una empresa fabrica dos tipos de prendas de vestir; camisetas y pantalones. El tiempo necesario para fabricar un pantalón es de 3h y se necesita 4m de tela. En el caso de las camisetas se necesitan 2h y son necesario 3m de tela. También se sabe que la tela disponible para una semana es de 160m y con los empleados que hay contratados se puede invertir 150h a la semana de trabajo, el dueño de la empresa nos ha pedido que a partir del costo de producción y el precio de venta de las prendas se estime cuantas camisetas y cuantos pantalones se tienen que fabricar para obtener el máximo de ganancias.

En la siguiente tabla de datos se muestre un resumen:

	Pantalón	Camiseta
Horas de Producción por prenda	3	2
Metros por prenda	4	3
Costo de producción	6	4
Precio Venta	12	8

Se busca representar el problema, para ello es necesario utilizar dos variables. Las llamaremos x1 al número de pantalones y x2 al número de camisetas. Lo que se busca es el máximo beneficio de los recursos con los que se dispone, así que se definirá el beneficio según la siguiente formula.

Beneficio = (precio de venta – costo de producción) * unidades vendidas

Como el beneficio total sera la suma de los beneficios de ambas prendas se llega la siguiente ecuación que representa la función de un beneficio para este caso en concreto quedaría de la siguiente manera:
$\int(x1, x2) = (12 – 6)x1 + (8 - 4)x2$

Por otro lado hay que poner una serie de restricciones. La primera y la mas obvia es que ambas variables han de ser mayores o iguales a 0. No se puede fabricar un numero negativo de prendas.
$x1 \geq 0$
$x2 \geq 0$
Tambien se sabe que se dispone de una capacidad de trabajo semanal de 150 h, como fabricar un panatalon se invierte 3 horas y dos horas de una camiseta, se ha de imponer la siguiente restriccion al problemas
$3x1 + 2x \leq 150$

Lo mismo sucede con la cantidad de metros de tela disponibles semanalmente, que son de 160 m, como un pantalón necesita 4 metros de teka y una camiseta 3, se impondra la siguiente restriccion:

$4x1 + 3x2 \leq 160$

En resume, el problema de PLE

$f(x1, x2) = (12 - 6)x1 + (8 - 4)x2$

Restricciones:

$3x1 + 2x2 \leq 150$
$4x1 + 3x2 \leq 160$
$x1 \geq 0$
$x2 \geq 0$

El objetivo es encontrar una asignación de valores enteros para las variables x1 y x2 que, cumpliendo con las restricciones impuestas, hagan que la función **f** tome el máximo posible.

se puede comprobar que la variable x1, este acotada entre 0 y 50, mientras que x2 lo esta entre 0 y 75. por lo tanto el espacio de estados constara de 50 *75 =3750 posibilidades, que no es un numero muy elevado. Es perfectamente tratable por un ordenador. Sin embargo,en este problema solo entran en juego dos variables y rango de valores posibles es muy limitado. Puede demostrarse que para mayor numero de variables y rango de valores del problema es intratable.

La función de evaluación del problema es muy evidente. Se puede usar la misma función que maximice y pueda comparar las diferentes asignaciones de variables. a mayor valor de la función, mejor sera la calidad de las funciones de las variables.

Un problema relacionado **PLE** es de la programación no lineal **(PNL)** en que la función o alguna de las restricciones es una función no lineal y las variables toman valores reales.

Otros Problemas

Los problemas propuestos hasta ahora no han sido escogidos al azar. Son problemas bien conocidos y estudiados, ya que son problemas canónicos; es decir, cada uno de estos problemas representan y se resuelven de la misma manera que una multitud de otros problemas diferentes. En definitiva, son un modelo que permite atacar la resolución de otros similares. Por ejemplo el TSP es la base de un problema con mucha mayor aplicación práctica: el problema de las rutas de vehículos o Vehicle Routing problema (VRP). Las distribuidoras de mercancías o las empresas de reparto tratan siempre de optimizar las rutas de reparto, ya que esto trae un gran beneficio económico por el ahorro en recursos: tiempo, combustible, vehículos y personal. El problema consiste en, dado un numero arbitrario de clientes a los que hay que servir mercancía, calcular las rutas que minimicen el número de vehículos necesarios y el número de kilómetros recorridos por cada vehículo. Al resolver el problema de la ruta de vehículos hay que tener en cuenta que las rutas pueden tener asociadas restricciones como:

- Carga máxima de los vehículos: dependiendo de la cantidad, peso, volumen de los pedidos de cada cliente, un vehículo podrá atender a más o menos clientes en un viaje.

- Horario de los clientes: las rutas tienen que diseñarse de acuerdo con los horarios de los clientes. De nada sirve que la ruta sea optima si el transportista, que suele cobrar por horas, tiene que esperar una hora a que habrá el comercio del cliente para poder entregar la mercancía.
- Horario del transportista: Los transportistas pueden limitarse a solo la distancia recorrida o el tiempo de trabajo que pueden realizar sin descansar.
- Preferencia: algunos clientes pueden ser más importantes que otros y por lo tanto es posible que se interese entregarles antes de los demás.
- Número de almacenes: hay que tener en cuenta que almacenes surten a cada cliente en concreto.
- Recarga: Otra condicionante es si los vehículos cargan la mercancía una vez o pueden volver al almacén a volver a cargar más mercancía.

Evidentemente, si se puede presidir de un vehículo, la empresa se ahorra un transportista y a menor número de kilómetros, mayor ahorro de combustible.

A lo largo de este curso se presentaran técnicas que permitan liderar con la complejidad implícita de estos problemas y resolverlos, o al menos conseguir aproximaciones lo suficientemente buenas para ser aceptadas.

Pero la IA trata de resolver muchos mas problemas y de distinta índole, estos son algunas técnicas de IA con resultado comprometedor.

- Minería de Datos
- Reconocimiento del Habla
- Reconocimiento de escritura
- Visión Artificial
- Diagnostico Medico
- Robótica
- Entornos industriales
- Planificación y control
- Videojuegos
- Seguridad (Rust)
- Predicciones Financieras
- Predicciones de Catástrofes
- Marketing especializado

BÚSQUEDA NO INFORMADA

Las técnicas que se presentan a continuación son conocidas de forma genérica como **búsqueda,** ya que pretenden encontrar una solución valida dentro de estados.

En concreto, las que se analizaran en este tema se denomina técnicas de **búsqueda no informada** debido a que el problema que se quiere resolver no ofrece ninguna información

adicional que ayude a encontrar una solución mas rápida, mas allá de la que proporciona el propio enunciado.

Retomemos el problema de encontrar la ruta mas corta por carretera entre dos ciudades. El problema proporcional de información sobre el riesgo de la búsqueda; es decir su función de evaluación que permite conocer la calidad de un estado en comparación a otros. Esto permite cierto margen de maniobra en la toma de decisiones durante la búsqueda para mejorar el proceso. Por ejemplo; supóngase que en el proceso de búsqueda se tienen a medio construir una ruta en el que por el momento actual tiene 1,200km (Todavía no es una solución completa porque la solución no llega al destino). Sin embargo, en una ruta de las analizadas anteriormente se había encontrado una ruta valida con 1,112km. En este caso, se puede abortar la búsqueda actual, ya que se sabe que sera peor que otra que ya se había evaluado anteriormente. Cuando se tiene información que ayuda a guiar la búsqueda de usan técnicas de **búsqueda informada** que serán tratadas mas adelante.

Como ejemplo para búsqueda no informada, imaginemos que se han perdido las llaves del coche, se sabe que están en una habitación de la casa, pero no se sabe nada mas. En este caso no queda mas remedio que realizar la búsqueda por toda la habitación hasta encontrarla. Ningún otro dato permite guiar la búsqueda. En este caso solo se usaran técnicas de búsqueda no informada, también conocida como búsqueda a ciegas.

El siguiente ejemplo ayudara a representar algunos conceptos importantes. Un Puzzle lineal es un puzzle para niños con grandes piezas, se llama puzzle lineal porque todas sus piezas solo pueden unirse en una linea como el siguiente ejemplo.

1	2	3	4

Realmente se podría sacar ventaja de algunas cuestiones, como por ejemplo, la forma de las piezas para probar algunas de las técnicas de búsqueda informada que se verán en el próximo tema. Sin embargo, en este ejemplo, se supone que todas las piezas son exactamente cuadradas y encajan una con otra sin importar el orden.

Antes de abordar la solución, hay que hacer algunas preguntas que ayuden a definir el problema.

- ¿Cuál es el espacio de estado del problema?
- ¿Cual es el estado inicial y final (Objetivo)?
- ¿Que operación nos permite pasar de un estado a otro?

1	2	3	4

En cuanto al espacio de estados, un estado puede ser cualquier ordenación posible de cuatro piezas, así que se tendrán 4!=24 posibles estados. Aunque como se ha analizado previamente se puede saber que a mayor numero de piezas se volvería problema intratable.

Es necesario definir una **operación** que permita ir explorando los posibles estados. Una operación básica es intercambiar dos piezas continuas. Para el caso del Puzzle de 4 elementos, se tendrán 3 operaciones posibles.

- Intercambiar las dos piezas de la derecha: Operación **D.**
- Intercambiar las dos piezas centrales: Operación **C.**
- Intercambiar las dos piezas de la izquierda: Operación **I.**

Por ultimo de necesita una operación de evaluación. La mas lógica seria que evaluara cuantas piezas hay, bien colocadas, pero como se quiere mostrar las técnicas de búsqueda no informada, es necesario obviar ese dato y mostrar la función de evaluación. Esto comprobara se se ha llegado al estado objetivo o no. en el siguiente tema se retomara este mismo problema para resolverlo usando técnicas de búsqueda informada.

REPRESENTACIÓN DE ESTADOS: ÁRBOLES Y GRAFOS

Ahora se sabe algo mas sobre el problema, se necesita plantear otras cuestiones referentes a la implementación para la solución en el ordenador. La primera y la mas básica es: como se representa el Puzzle y sus estados en un programita? Parece una buena idea representar el puzzle como un array de números enteros con una dimensión de 4. Usando esta representación algunos posibles estados podrían ser:

[2,1,4,3]
[1,2,4,3]
[4,2,3,1]

Cualquiera de ellos es valido. Si se aplica cada uno de los operadores que se han definido (I,C,D) los arreglos se transformaran de la siguiente manera:

[2,1,4,3] [1,2,4,3] Operador I →
[1,2,4,3] [1,4,2,3] Operador C →
[4,2,3,1] [4,2,3,1] Operador D →

La forma de representar los estados y el resultado de aplicar un operador es un árbol que muestra como evolucionan los estados al ir aplicando los operadores que se han definido.

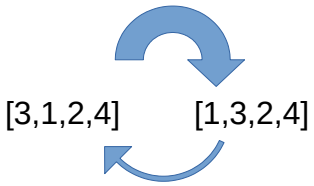
TAREA: generar la complejidad de estados partiendo del 1,2,3,4 (24)
[1,2,3,4] (Estado Inicial)
[2,1,3,4],[1,3,2,4],[1,2,4,3]
[2,3,1,4],[2,1,4,3],[3,1,2,4],[1,3,4,2],[1,4,2,3]
[3,2,1,4],[2,3,4,1],[2,4,1,3],[3,1,4,2],[1,4,3,2],[4,1,2,3]
[3,2,4,1],[2,4,3,1],[4,2,1,3],[3,4,1,2],[4,1,3,2]
[3,4,2,1],[4,2,3,1],[4,3,1,2]
[4,3,2,1] (Estado más alejado o "invertido")

Dentro de la nomenclatura usada en la teoría de árboles y grafos, se llamarán **nodo** a cada uno de los estados. A los nodos resultantes después de aplicar los operadores se le llamaran **hijos** y al nodo que generan los hijos se les llamará **Padre.** Todos los nodos hijos del mismo padre se le denominan **hermanos.** El nodo superior es el único que no tiene padre, se le llamará nodo **raíz.** En este caso se corresponde con el estado inicial [3,1,2,4]. a un nodo que no tiene hijos se le llamará **hoja.** Un nodo que no tenga padre y al menos un hijo se le llamará **rama.**

Como se tienen tres operadores, al aplicarlo a un nodo, en este caso se dice que el árbol tiene un **factor de ramificación** de 3, ademas cada vez que se aplica un operador se dice que se ha descendido un **nivel** en la jerarquizar del árbol.

Si se observa el desarrollo del árbol, puede comprobarse que hay estados que son iguales.
Por ejemplo: si el estado inicial aplica el operador D y únicamente se vuelve a aplicar de nuevo se obtiene el estado inicial, en este caso se pueden moldear los estados como un grafo, que es una estructura similar al árbol pero que tiene dos ciclos. Los grafos suelen representarse uniendo sus nodos con lineas o flechas curvas si se puede pasar de un estado a otro en ambos sentidos se usara una linea. Sin embargo la transición de un estado a otro solo puede darse en un sentido, se usara una flecha para iniciarlo.

Cuando el desarrollo se encuentra en ciclos, como ha pasado en este caso, la busqueda de la solucion podria no encontrarse nunca, ya que el programa podria quedar indefinidamente en esos ciclo. Por lo que se tendra que evitarlos, una fomra habitual de hacerlo es “recordar” que nodo se han visitado almacenándolos en memoria y cada vez que se llegue a un nuevo nodo, comprobar que se ha visitado con anterioridad.



Regresa al mismo estado en bucle***

Encontrar una solucion a un problema consistirá en hacer una búsqueda en el árbol de estados o en un grafo para encontrar un nodo que contenga un estado objetivo, en general el recorrido de un árbol(grafo), se compone de los siguientes pasos:

- Seleccionar el nodo raiz en una lista que contendra aquellos nodos que estan pendientes de visitar. A esta lista se le llamara **frontera** (Tambien se le conoce como lista abierta)
- Seleccionar un nodo de la lista nodos frontera y comprobar si es un nodo objetivo, Si lo es, se ha terminado.
- Generar todos los hijos del nodo seleccionado en paso 2, aplicando los operadores que se han definido.(Esto es lo que se llama expandir un nodo, añadir los nuevos nodos hijo a la lista de nodos frontera)

- Volver al paso 2 hasta que la lista de nodos frontera este vacía.

Si en lugar de un arbol se esta trabajando con un grafo, el proceso cambia ligeramente

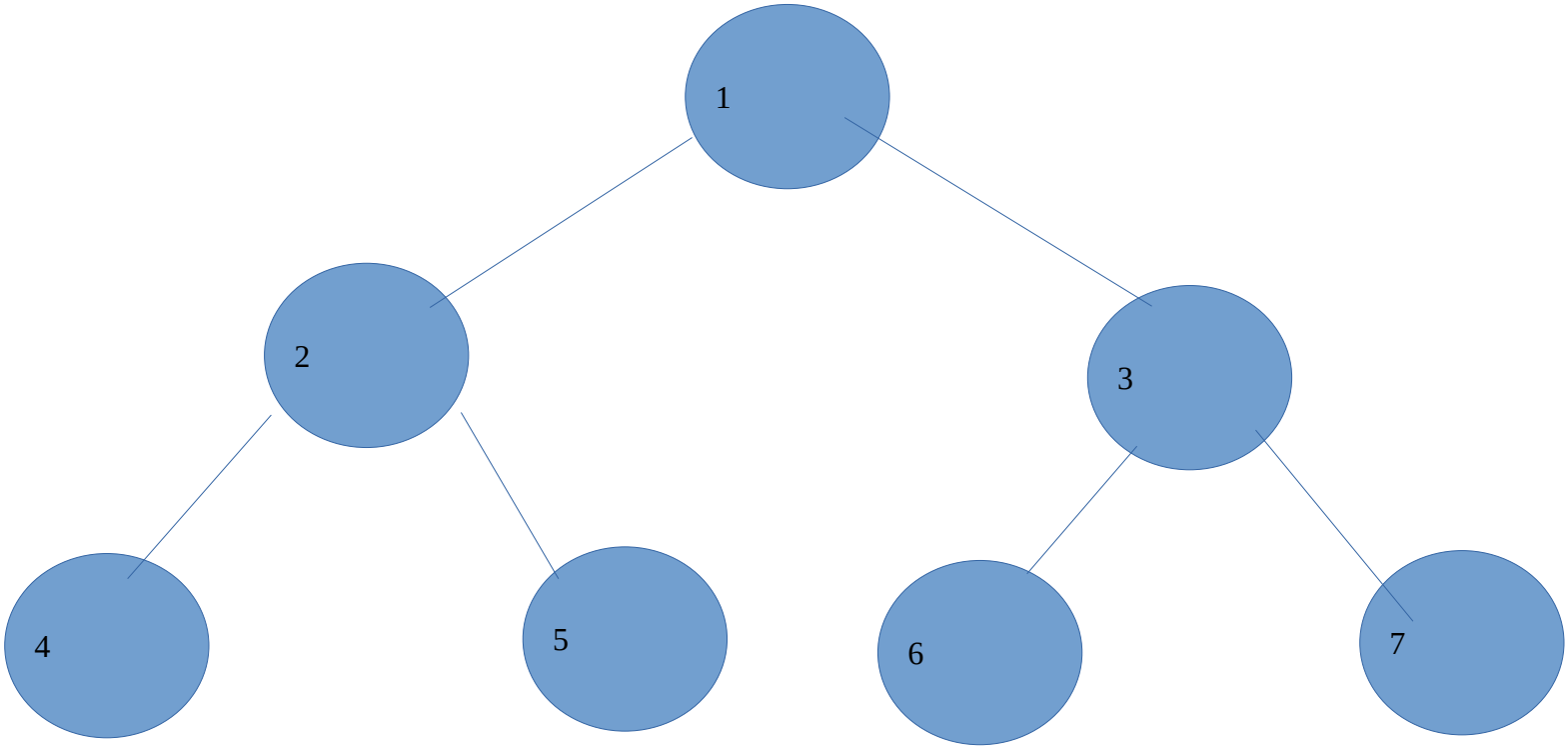
1. Seleccionar el nodo raiz y almacenarlo en la lsita de nodos frontera
2. Seleccionar un nodo de la lista de nodos frontera y comprobar si es un nodo objetivo, si lo es se ha terminado ademas almacenamos el nodo en una lista llamada “visitados” y si no esta añadirlo a la lista de nodos frontera.
3. Generar todos los nodos hijo del nodo seleccionado en el paso 2, aplicando los operadores que se han definido. Para cada hijo comprobar que no este en la lista de nodos visitados y si no esta añadirla a la lista de nodos frontera
4. Volver al paso 2 hasta que la lista de nodos frontera este vacía.

Busqueda en amplitud

El proceso de búsqueda que se acaba de presentar es un procedimiento generico que puede dar lugar a variaciones, La primera que se va a presentar se llama búsqueda en amplitud o Breadth First Search(BFS), Esta busqueda recorre el arbol por niveles, Es decir.

- Primero se visita el nodo raiz
- Seguidamente se visitan todos sus hijos
- Para cada hijo en el paso anterior, se visitan todos sus hijos y asi sucesivamente.

Este recorrido se implementa en la practica usando una cola (FIFO) First in First out para la lista de nodos frontera, una cola FIFO es una estructura de datos en la que se puede hacer 2 operaciones principales. Almacenar un dato y scar un dato. La particularidad de las colas FIFO es que cuando se saca un dato este se extrae en el mismo orden. A continuacion se muestra el algoritmo se busqueda



A continuacion se muestra el algoritmo de busqueda en amplitud en pseucodigo:

Nodo_inicial = estado inicial
Nodos_frontera = Cola FIFO
nodos_visitados = lista
Almacenar Nodo_iicial en nodos_frontera
Mientras los nodos_frontera no se encuentren vacio:
    Nodo_Actual = Extraer nodo_frontera
    Si nodo_actual == solucion:
        Salir con la solucion
    Introducir nodo actual en nodos visitados
    Por cada operador:
        Nodo hijo= operador(nodo-actual)
        Si nodohijo no esta en nodos vistados ni nodos_frontera:
            introducirnodo_hijo en nodos_frontera

Hay que llevar a la practica el algoritmo haciendo un programa para resolver el problema del puzzle lineal, a lo largo del curso será necesrio usar la clase nodo es una sencilla implementacion de la estructura de datos del arbol de la que se ha hablado en esta seccio. El archivo se llamara arbol.py

```
class Nodo:
def __init__(self, datos, hijos = None):
self.datos = datos
self.hijs = None
self.padre = None
self.costo = None
self.set_hijos(hijos)

def set_hijos(self, hijos):
self.hijos= hijos
if self.hijos != None:
for h in self.hijos:
h.padre = self

def get_hijos(self):
return self.hijos
def get_padre(self):
return self.padre
def set_padre(self, padre):
self.padre = padre

def set_datos(self, datos):
self.datos = datos

def get_datos(self):
return self.datos
```

```
def set_costo(self, costo):
self.costo = costo

def get_costo(self):
return self.costo
def igual(self, nodo):
if self.get_datos()== nodo.get_datos():
return True
else:
return False
def en_lista(self, lista_nodo):
en_la_lista = False
for n in lista_nodo:
if self.igual(n):
en_la_lista = True
return en_la_lista
def __str__(self):
return str(self.get_datos)
```

igual(nodo) → Devuelve True si el dato contenido en el nodo es igual a alguno de los nodos contenidos en la lista de nodos pasada como parametros.

El codigo del programa que resuelve el problema del puzzle lineal es el siguiente: